

LED Lab 3

Welcome to the third lab in our series on Light-Emitting Diodes (LEDs)! Building on the foundational concepts of basic output control from the previous labs, this project introduces a more advanced technique for managing time in a microcontroller. We will create an engaging pattern where two pairs of same colored LEDs blink, with one pair turning on as the other turns off.

The primary objective of this lab is for students to:

- **Work with multiple output pins simultaneously, controlling four LEDs.**
- **Understand the need for and implement non-blocking timing using the millis() function, which is critical for complex microcontroller projects.**
- **Practice using Boolean variables (bool) and comparison operators (>=) for conditional logic.**

Lab 3 – 4-Way Alternating LED Blinker

In the previous lab we utilized the delay() function to program the stop time for the microcontroller's execution. Well in this lab, we have designed our code to avoid using the delay() function, which completely stops the microcontroller's execution. Instead, we use the millis() function, which keeps track of the time passed since the program started, allowing the microcontroller to do other tasks while waiting for a blink interval to pass.

Enter the Code

Open your Arduino IDE and type in the code below.

```
//Define the LED pins in the code

int LED_1 = 2;  //LEDs 1 and 2 will blink together

int LED_2 = 4;

int LED_3 = 6;  //LEDs 3 and 4 will blink together

int LED_4 = 8;


// Variables to track timing without blocking

unsigned long previousMillis = 0;


// Blink interval (milliseconds)

const long interval = 1000; // Switch between pairs every 500ms
```

```

// LED states - only one pair on at a time

bool pair1Active = true; // Start with pair 1 active


void setup() {

    // Set LED pins as outputs

    pinMode(LED_1, OUTPUT);

    pinMode(LED_2, OUTPUT);

    pinMode(LED_3, OUTPUT);

    pinMode(LED_4, OUTPUT);

}


void loop() {

    unsigned long currentMillis = millis();


    // Switch between pairs every interval

    if (currentMillis - previousMillis >= interval) {

        previousMillis = currentMillis;

        pair1Active = !pair1Active; // Toggle active pair


        if (pair1Active) {

            // Turn on LEDs 1&2, turn off LEDs 3&4

            digitalWrite(LED_1, HIGH);

            digitalWrite(LED_2, HIGH);

            digitalWrite(LED_3, LOW);

            digitalWrite(LED_4, LOW);

        } else {

            // Turn on LEDs 3&4, turn off LEDs 1&2

            digitalWrite(LED_1, LOW);

            digitalWrite(LED_2, LOW);

            digitalWrite(LED_3, HIGH);

            digitalWrite(LED_4, HIGH);

        }

    }

}

```

Mini Lab: 4-Way Alternating LED Blinker (Code Overview)

Function and Hardware Definitions

```
int LED_1 = 2;
```

```
int LED_2 = 4;
```

```
int LED_3 = 6;
```

```
int LED_4 = 8;
```

These lines define which digital pins on the Arduino are connected to each of the four LEDs.

Timing Variables:

```
unsigned long previousMillis = 0;
```

This variable stores the time (in milliseconds) when the LEDs were last switched. We use the **unsigned long** data type because the number of milliseconds can become very large over time (up to 50 days) and must always be positive.

```
const long interval = 1000;
```

This constant defines how long to wait (1,000 milliseconds or 1 second) before switching the LED pairs.

```
bool pair1Active = true;
```

This is a **Boolean variable** that holds only one of two possible values: true or false. It tracks which pair of LEDs is currently supposed to be ON.

```
Setup() {
```

```
  pinMode(LED_1, OUTPUT);
```

```
  pinMode(LED_2, OUTPUT);
```

```
  pinMode(LED_3, OUTPUT);
```

```
  pinMode(LED_4, OUTPUT);
```

```
}
```

The setup() function runs once when the program starts. It simply configures all four LED pins to act as **outputs**, allowing the Arduino to send electricity through them.

Void loop()

The loop() function contains the core logic that runs repeatedly

unsigned long currentMillis = millis();

The built-in millis() function returns the number of milliseconds that have passed since the Arduino board began running the program. This value is saved to currentMillis.

if (currentMillis - previousMillis >= interval)

This is the heart of **non-blocking timing**. It checks if the difference between the currentMillis and the previousMillis (the time of the last switch) is greater than or equal to the desired interval (1000ms).

If the required time has not passed, the code inside the if block is skipped, and the program quickly repeats the loop, ready to execute any other code you might add later.

Perform Switch (If Time Has Passed):

previousMillis = currentMillis;

If the condition is met, we reset the timer by saving the current time as the new time of the last switch.

pair1Active = !pair1Active;

The ! operator means **NOT**. This toggles the state of the Boolean variable. If **pair1Active** was true, it becomes false, and if **pair1Active** was false, then it becomes true.

```
if (pair1Active) {  
    digitalWrite(LED_1, HIGH);  
    digitalWrite(LED_2, HIGH);  
    digitalWrite(LED_3, LOW);  
    digitalWrite(LED_4, LOW);  
}  
else {  
    digitalWrite(LED_1, LOW);  
    digitalWrite(LED_2, LOW);  
    digitalWrite(LED_3, HIGH);  
    digitalWrite(LED_4, HIGH);  
}  
}
```

The if/else blocks checks the new state of **pair1Active** and sets the output pins accordingly:

If true, LEDs 1 and 2 are set HIGH (ON), and 3 and 4 are set LOW (OFF).

If false, LEDs 3 and 4 are set HIGH (ON), and 1 and 2 are set LOW (OFF).

Now lets look into Exploring Timing Changes in this lab

Your current code uses the interval variable to control how fast the LEDs switch.

Changing the Blink Speed

To change the blink speed, you only need to modify one line of code:

// Blink interval (milliseconds): Switch between pairs every 1000ms (1 second)

const long interval = 1000;

- To Make it Faster (e.g., half a second): Change the value from 1000 to 500.

const long interval = 500;

- To Make it Slower (e.g., 2 seconds): Change the value from 1000 to 2000.

const long interval = 2000;

Key Learning Point: Because you used the interval variable, you didn't have to change multiple `delay()` lines throughout the code (which is a common mistake in beginner projects). This is a great demonstration of how using **variables** makes your code more efficient and maintainable.

Mini Lab: 4-Way Alternating LED Blinker (Hardware Overview)

Since this project uses four LEDs, we need four separate current-limiting resistors. The calculations for the resistor value remain the same as in Lab 2

Component	Quantity	Key Function
Arduino UNO R3	1	The main programmable controller.
Breadboard	1	Used to connect all components without soldering.
5mm LED	4	Converts electrical energy into light.
100-ohm Resistor	4	Limits the current to protect each LED from burning out.
Jumper Wires	10	Used to connect the four LEDs and resistors to the Arduino pins.

Connecting Everything (Step by Step Instructions)

Before you begin wiring, remember that each LED **must** have its own current-limiting resistor. We will be using four 100 Ω resistors, one for each LED.

Follow these steps carefully to build the circuit and upload the code:

Step 1: Place and Wire the Components

1. **Place the LEDs:** Insert the four 5mm LEDs into the breadboard. Remember the longer leg (Anode or positive side) must be connected to power, and the shorter leg (Cathode or negative side) to the ground path.
2. **Attach the Resistors:** Connect one lead of a 100 Ω resistor to the shorter leg (Cathode) of each of the four LEDs.
3. **Connect to Ground (GND):** Use jumper wires to connect the other lead of all four resistors together and then connect this common point to one of the **GND** (Ground) pins on your Arduino board. This completes the negative side of the circuit.
4. **Connect to Digital Pins:** Use jumper wires to connect the longer leg (Anode) of each LED to the specified digital pins on the Arduino:

LED 1 (Pair 1): Connect to Digital Pin 2.

LED 2 (Pair 1): Connect to Digital Pin 4.

LED 3 (Pair 2): Connect to Digital Pin 6.

LED 4 (Pair 2): Connect to Digital Pin 8.

5. **Connect the Arduino:** Plug your Arduino UNO board into your computer using the USB cable.
6. **Select Board and Port:** Ensure the correct board type and COM port are selected in the Arduino IDE **Tools** menu.
7. **Upload:** Click the **Upload** button to transfer the code to your Arduino board.
8. **Observation:** If the circuit is wired correctly and the code is successfully uploaded, you should observe the LEDs blinking in an alternating pattern:

LEDs 1 and 2 will turn **ON** together.

After 1 second (1000ms), LEDs 1 and 2 will turn **OFF** and LEDs 3 and 4 will turn **ON**.

This pattern will repeat indefinitely, demonstrating non-blocking timing.